Brief Announcement: What is Agreement About if not Common Knowledge?

Or David
Technion
Haifa, Israel
ordavid90@gmail.com

Yoram Moses
Technion
Haifa, Israel
moses@technion.ac.il

CCS Concepts

• **Theory of computation → Distributed algorithms**.

Keywords

Agreement problems and protocols, all-or-nothing decision, fault tolerance, common knowledge, dynamic common knowledge

ACM Reference Format:
Or David and Yoram Moses. 2026. Brief Announcement: What is Agreement About if not Common Knowledge?. In *ACM Symposium on Principles of Distributed Computing (PODC '26), July 6–10, 2026, Egham, United Kingdom.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3796701.3815934

1 Introduction

Agreement problems lie at the heart of distributed computing, and have deservedly received significant attention by the research community over the last fifty years. Experts have developed a large toolbox of *tricks of the trade* for solving agreement problems. Fairly general proof techniques [2, 8, 15, 16], including tools based on topology [22–24] and ones based on reasoning about knowledge [11, 13, 29], have been developed and applied to the study of agreement problems.

This paper presents a unifying approach to the study of a family of agreement problems that are central to fault-tolerant distributed computing, in which the concern is that nonfaulty processes all end up acting in the same manner.[1] More precisely, a process i can perform an action of "deciding" on a value v (which we denote by $decide_i(v)$) and in any given execution, if any nonfaulty process i performs $decide_i(v)$, then every nonfaulty process j performs $decide_j(v)$ in this execution. We call such behavior *all-or-nothing* decision (AoN, for short). Consensus and Byzantine agreement [10, 28, 31], weak Byzantine Agreement [25], Atomic commitment [12, 20], Reliable broadcast [4],[2] Firing Rebels with Relay [17] and Coordinated attack[1, 19] are all well-known agreement problems satisfying AoN, whose study has generated a vast literature. In agreement problems that do not imply AoN, the decisions of nonfaulty processes are typically required to be compatible, but some variation among their decisions is allowed. These include k-set Consensus [6], Crusader agreement [8], Stabilizing Consensus [5] and Approximate agreement [9]. We remark that the Paxos protocol of[26] and many of its variants also satisfy AoN. Moreover, some popular randomized solutions to Consensus do satisfy AoN (e.g., Ben Or [3]), while others (e.g., Feldman and Micali [14]) do not.

Our approach to the study of AoN agreement is rooted in Knowledge theory [13, 21], which is a methodology that relates actions in a distributed protocol to knowledge, or information, that the process performing an action must have. In its pure form, agreement entails common knowledge. Thus, for example, once Agnes and Reese have agreed to meet at Noon for lunch at the cafeteria, this implies that both of them know that they have agreed. Given that this is an inherent aspect of their agreement, one can infer that they both know that they know they have agreed. Moreover, this reasoning can be extended to show that they have arbitrarily deep levels of knowledge about their lunch plans. The state of knowledge in which a set of agents has arbitrarily nested levels of everyone knowing that everyone knows was named *common knowledge* by the philosopher David Lewis in his book on conventions [27]. In a later work, Halpern and Moses in [21] formally proved that new facts such as a plan to meet at Noon can only become common knowledge *simultaneously*. This suggests that in many settings of interest common knowledge cannot be attained. Consequently, achievable forms of agreement will necessarily satisfy slightly weaker properties.

Note that the specifications of most popular distributed agreement problems do not require decisions to be simultaneous. In a recent paper [18], Gonczarowski and Moses introduced *dynamic common knowledge*, which is a variant of common knowledge that arises naturally without requiring simultaneity. In the current work we adapt dynamic common knowledge to a fault-tolerant distributed setting by defining dynamic common knowledge among the *nonfaulty* processes in a given run. As we show, in all AoN agreement problems, before it can decide v a process must know that there is dynamic common knowledge of several facts, including: (i) that the decision value in the current run is v, (ii) that v is a *valid* value, i.e., the specification allows deciding on v in the current run, and (iii) that no nonfaulty process decides on a conflicting value $w \neq v$. This implies that every AoN agreement protocol must include a mechanism to ensure that these facts become dynamic common knowledge, and must provide means for detecting that a process knows about it.

The Logic of Knowledge. Our analysis will make use of the runs and system modeling of knowledge in distributed systems (see [13, 29]). We now present just enough of the framework to state and discuss our results. We typically reason about some protocol P

[1] The definition of "faulty" and "nonfaulty" is model dependent (e.g. crash, byzantine). The specification of agreement problems typically leaves the explicit operational definition of what it means to be faulty unspecified, and this is also the approach we shall adhere to.
[2] For uniformity, we refer to "delivering" a message in reliable broadcast as "deciding" on the message.



This work is licensed under a Creative Commons Attribution 4.0 International License.
PODC '26, Egham, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2512-8/26/07
https://doi.org/10.1145/3796701.3815934

in a given model of computation, and will consider the set of runs of P in that model as determining all possible runs. We typically denote facts by Greek letters, such as φ, ψ etc. We write $(r,t) \models \varphi$ to state that the fact φ is true at time t in the run r.[3] Specific facts of interest in this brief announcement are $decide_i(v)$, which is true at points where i decides v. We will assume that every run r determines a set of *nonfaulty* (i.e., correct) processes (denoted $\mathbf{N}(r)$), and use '$i \in \mathbf{N}$' as the fact stating that i is nonfaulty in the current run. Finally, we write $\texttt{all_decide}(v)$ to denote the fact that is true in a run r if every nonfaulty process $i \in \mathbf{N}(r)$ decides v in the run (not necessarily all at the same times). The latter two formulas are instances of *facts about runs*, whose truth at a point (r,t) depends only on r and not on t. For such facts, we sometimes write $r \models \psi$ to state that $(r,t) \models \psi$ holds for all $t \geq 0$. Finally, a fact φ is called *valid*, denoted by $\models \varphi$, if $(r,t) \models \varphi$ holds for all points (r,t). Formulas are closed under the Boolean operators $\wedge, \neg, \Rightarrow$ and for every formula φ, we denote by $K_i\varphi$ the fact that "process i knows φ." By definition, $(r,t) \models K_i\varphi$ iff for all points (r',t') at which i has the same local state as at (r,t) we have $(r',t') \models \varphi$. We call a fact ψ *local to* i if, for all points (r,t), its truth at (r,t) depends only on i's local state at that point. Knowledge formulas are in particular i-local, and the knowledge axiom $\models K_i\varphi \Rightarrow \varphi$ is always valid.

Knowledge of Preconditions for decision actions. We shall assume for simplicity that protocols determine whether to perform a decision action based on the local state of a process. Moreover, if a nonfaulty process is in a state where it is about to decide v, it will remain in such a state until it performs the action. Decisions are thus *conscious* actions, in the language of [29]. In fault-tolerant distributed computing, a faulty process does not always know whether or not it is faulty. When processes fail by crashing, for example, a process might not know whether it will crash in the future (and is thus in fact faulty in the current run). An instance of the *knowledge of preconditions principle* theorem in [29] is that if ψ is a precondition for performing the action $decide_i(v)$, then so is $K_i(i \in \mathbf{N} \Rightarrow \psi)$. We remark that Moses and Tuttle in [30] considered $K_i(i \in \mathbf{N} \Rightarrow \psi)$ as a useful notion of "belief" in the analysis of Consensus. In fact, for nonfaulty processes, such beliefs are never false:

$$\models \ \big(i \in \mathbf{N} \ \wedge \ K_i(i \in \mathbf{N} \Rightarrow \psi)\big) \ \Rightarrow \ \psi.$$

This motivates the following

Definition 1.1 (Nonfaulty knowledge). We use the notation $K_i^N\varphi$ as shorthand for $\big(i \in \mathbf{N} \wedge K_i(i \in \mathbf{N} \Rightarrow \varphi)\big)$ and refer to the K_i^N operator as "i's nonfaulty knowledge."[4]

An immediate application of the results in [29] provides the following *knowledge of preconditions principle* for nonfaulty processes in decision protocols:

Theorem 1.2 ([29]). *For all facts ψ,*

If $\models \big(i \in \mathbf{N} \wedge decide_i(v)\big) \Rightarrow \psi$

Then $\models \big(i \in \mathbf{N} \wedge decide_i(v)\big) \Rightarrow K_i^N(\psi)$.

[3] This definition depends on the set of runs of the protocol P in question in the model of computation of interest; we omit these from the notation for ease of exposition.

[4] Clearly, K_i^N satisfies the knowledge axiom $\models K_i^N\varphi \Rightarrow \varphi$. We note that for correct processes i, the operator K_i^N coincides with the *hope* operator H_i introduced by Kuznets *et al.* (see [17, 24]). For faulty processes i, the fact $K_i^N\varphi$ is never true, while $H_i\varphi$ holds for all facts φ (including total contradictions) at all times.

An instance of Theorem 1.2 is that when a correct process decides v, it must know that deciding v is allowed according to the problem specification. In Consensus, for example, this process must have nonfaulty knowledge that v satisfies Validity: at least one of the initial values in the current run was v.

Dynamic Common Knowledge. Our purpose is to relate AoN agreement protocols to the notion of dynamic common knowledge proposed in [18]. But in order to do so, we need to modify the definition to accommodate the fact that our systems can contain faulty processes, whose behavior and knowledge can be affected by the adversary. The essential change that leads to their new definition is that formulas can refer to what a process knows *when a local event of interest occurs at the process*. Thus, for example, When Agnes sends a package with a book to Reese (a local event for Agnes) she can know that, *when Reese receives it*, Reese will know the book title. In the formal definition to come, we will make use of two temporal operators, $\Diamond$ and $\boxdot$, the first stating that a fact occurs at *some* point in time in the current run, and the latter stating that a fact occurs at *all* points. Thus, $r \models \Diamond\varphi$ holds if there exists a time $t \geq 0$ such that $(r,t) \models \varphi$. Conversely, $r \models \boxdot\varphi$ holds if $(r,t) \models \varphi$ for all times $t \geq 0$.

Definition 1.3 (Dynamic Knowledge). For a process i, an i-local fact ψ_i, and a fact φ, we define the run-based fact $K^N_{i@\psi_i}(\varphi)$ as follows:

$$r \models K^N_{i@\psi_i}(\varphi) \qquad \text{iff} \qquad r \models \big(\Diamond\psi_i \ \wedge \ \boxdot(\psi_i \Rightarrow K_i^N\varphi)\big).$$

$K^N_{i@\psi_i}$ captures "dynamic knowledge" of a fact when the i-local event captured by ψ_i holds. This requires ψ_i to occur at least at one point in the run, and that i have nonfaulty knowledge of φ whenever ψ_i holds in the run.

Based on dynamic knowledge, we define dynamic variants of everyone knows and common knowledge, restricted to the nonfaulty processes. Assuming that the processes in the system are named $1, 2, \ldots, n$, these states of knowledge are defined with respect to a *local profile*, which is a list $\boldsymbol{\psi} = (\psi_1, \ldots, \psi_n)$ such that ψ_i is an i-local fact for every process $i = 1, \ldots, n$. The dynamic knowledge of every individual nonfaulty process will be evaluated relative to its respective local fact in $\boldsymbol{\psi}$.

Definition 1.4 (Dynamic Common Knowledge). We define the dynamic version of "every nonfaulty process knows" wrt. $\boldsymbol{\psi}$ by

$$r \models E_{N@\boldsymbol{\psi}}(\varphi) \qquad \text{iff} \qquad r \models K^N_{i@\psi_i}(\varphi) \ \text{ holds for all } i \in \mathbf{N}(r).$$

We denote by $E^m_{N@\boldsymbol{\psi}}(\varphi)$ the formula $E_{N@\boldsymbol{\psi}}(E_{N@\boldsymbol{\psi}}(\cdots(\varphi)))$ where '$E_{N@\boldsymbol{\psi}}$' is nested m times. As for standard common knowledge, we then define dynamic common knowledge by iterating everyone knows:

$$r \models C_{N@\boldsymbol{\psi}}(\varphi) \qquad \text{iff} \qquad r \models E^m_{N@\boldsymbol{\psi}}(\varphi) \ \text{ holds for all } m \geq 1.$$

Notice that all of the "dynamic" states of knowledge that we have just defined are facts about runs: A process can have (throughout the run) dynamic knowledge of a fact that it will learn only at time 1000. Nevertheless, it *is* guaranteed to know this fact at some point. Similarly, dynamic common knowledge cannot change over time. But if a fact is dynamic common knowledge, then every correct process will, at least at one point, know that it is. We are now ready to relate the new states of knowledge to fault-tolerant protocols.

All or Nothing Decisions. As mentioned earlier, we focus on problems and protocols in which in every run, either all nonfaulty processes decide v, or none do. Formally, a protocol satisfies the *All-or-Nothing* (AoN) decision property wrt. a given value v if:

$$\mathsf{AoN}(v): \quad \models \ \big(i \in \mathbf{N} \wedge \mathit{decide}_i(v)\big) \Rightarrow \mathtt{all_decide}(v).$$

It is straightforward from Theorem 1.2 that $\mathsf{AoN}(v)$ implies for correct i that $K_i^N\big(\mathtt{all_decide}(v)\big)$ is a precondition for deciding v. We can, however, show that, in fact, knowing that v is the decision value is *dynamic common knowledge* and is a precondition for deciding v. We denote by $\boldsymbol{\delta}(v) = (\delta_1(v), \ldots, \delta_n(v))$ the profile where for all i, $\delta_i(v) \triangleq \mathit{decide}_i(v)$. We can show:

Theorem 1.5 (Necessity of DCK). *If the protocol satisfies* $\mathsf{AoN}(v)$,

$$\models \ \big(i \in \mathbf{N} \wedge \mathit{decide}_i(v)\big) \Rightarrow K_i^N C_{N@\boldsymbol{\delta}(v)}\big(\mathtt{all_decide}(v)\big).$$

In addition to Agreement or AoN decision, problems often specify *validity* conditions, which are safety conditions that do not explicitly refer to decisions at different processes. Thus, for example, in Consensus, v satisfies Validity (which we denote by $\mathtt{valid}(v)$) if it is the initial value of at least one process. In Reliable broadcast, $\mathtt{valid}(v)$ (i.e., deciding—or delivering—the message v is allowed) holds unless the initiator is correct, and it initiated a value $w \neq v$. An immediate corollary of Theorem 1.5 is that, for every precondition φ_v for deciding v, when a correct process decides v, it must know that φ_v is dynamic common knowledge wrt. $\boldsymbol{\delta}(v)$ when deciding v. In particular, we have:

Corollary 1.6. *Let P be a protocol with decisions in V that satisfies* $\mathsf{AoN}(v)$ *for all $v \in \mathsf{V}$. Then*

- *If P satisfies a validity condition then*

  $$\models \ \big(i \in \mathbf{N} \wedge \mathit{decide}_i(v)\big) \Rightarrow K_i^N C_{N@\boldsymbol{\delta}(v)}\big(\mathtt{valid}(v)\big).$$

  Moreover,
- *Let* $\mathtt{no_dec}(w)$ *denote the fact "no correct process decides w" (in the current run). If P satisfies the Agreement condition, $v, w \in \mathsf{V}$ and $v \neq w$, then*

  $$\models \ \big(i \in \mathbf{N} \wedge \mathit{decide}_i(v)\big) \Rightarrow K_i^N C_{N@\boldsymbol{\delta}(v)}\big(\mathtt{no_dec}(w)\big).$$

Theorems 1.5 and 1.6 apply to *all protocols* that satisfy AoN, in *all models*: all network topologies, modes of communication among processes (message passing, shared memory, RDMA etc.), types and numbers of failures and all ratios between the number of possible failures and the number of processes. Note that the need to attain dynamic common knowledge follows already from the $\mathsf{AoN}(v)$ decision condition. These results establish that every protocol that solves an AoN agreement problem must provide a mechanism for guaranteeing that dynamic common knowledge is attained as a precondition for decision. The ways in which dynamic common knowledge can be achieved, and the facts that can become dynamic common knowledge clearly depend on the model under consideration. Studying ways of guaranteeing dynamic common knowledge promises to provide extensive insights into agreement problems and their solutions.

While Theorems 1.5 and 1.6 establish that dynamic common knowledge is a crucial necessary component of protocols solving AoN agreement problems, we can show a precise sense in which dynamic common knowledge can also be used as a sufficient condition and help in the design and optimization of agreement protocols. As motivation for our sufficiency theorem, let us consider the simple case of binary Consensus in a particular model. The runs of any protocol solving this problem can be partitioned into two: the runs in which 0 is decided and the ones in which 1 is decided. Denote by θ_0 the fact that the nonfaulty processes decide 0 in the run, and by θ_1 the fact that they decide 1. By Theorem 1.5 we have that $\models \theta_v \Rightarrow C_{N@\boldsymbol{\delta}(v)}(\theta_v)$, for $v = \{0, 1\}$. It follows that the protocol design involves determining such facts θ_0 and θ_1 that imply that 0 (resp. 1) is valid and are inherently dynamic common knowledge, as well as specifying the local conditions under which the decisions can be made.[5] More generally, we can prove the following converse to Theorems 1.5 and 1.6:

Theorem 1.7 (DCK is sufficient for agreement). *Let P be a protocol solving an* AoN *agreement problem A over a set V_A of values in a given model. Moreover, let $\Theta = \{\theta_v\}_{v \in \mathsf{V}_A}$ be a set containing a fact θ_v about runs (of P in this model) for each value $v \in \mathsf{V}$. If the following conditions hold for all $\theta_v, \theta_w \in \Theta$:*

1. *Θ-**Validity:*** $\models \theta_v \Rightarrow \mathtt{valid}(v)$ *(wrt. A);*
2. *Θ-**Exclusivity:*** $\models \theta_v \Rightarrow \neg\theta_w$ *for all $w \neq v$ in V_A; and*
3. *Θ-**Epistemic Sufficiency:** There exist profiles $\boldsymbol{\psi}(v)$ such that*
   - *$\models \theta_v \Rightarrow C_{N@\boldsymbol{\psi}(v)}(\theta_v)$, for all $v \in \mathsf{V}_A$, and*
   - *Nonfaulty processes remember being in $\boldsymbol{\psi}(v)$. More formally, for all runs r, nonfaulty processes $i \in \mathbf{N}(r)$ and all times $t' > t \geq 0$, if $(r, t) \models \psi_i(v)$ then $(r, t') \models \psi_i(v)$.*

Then there exists a protocol P_A that extends P by adding new decision actions such that, with respect to these new decisions, P_A is an AoN *protocol satisfying both Agreement and A's validity condition. Moreover, for every run r of P_A and $v \in \mathsf{V}_A$ the nonfaulty processes decide v in r iff $r \models \theta_v$. Finally, if $\models \bigvee_{v \in \mathsf{V}_A} \theta_v$, then P_A satisfies Decision.*

As with our previous results, Theorem 1.7 holds for the large and important class of AoN agreement problems in all possible models. In the full paper we illustrate ways in which various echoing mechanisms can be used to guarantee dynamic common knowledge of facts of interest in popular models. This includes how small committees, as in Algorand [7], and how accounting for failures, can improve the complexity of attaining dynamic common knowledge.

Underlying the design of every AoN protocol is a set of facts Θ as described in Theorem 1.7. Identifying this set provides insight into the structure of the particular protocol, and often facilitates modifying the protocol, thereby improving the time and communication complexities in runs of interest. In an analysis that is beyond the scope of a PODC Brief Announcement, we illustrate this by considering a Consensus protocol in the crash failure model, and studying Bracha's asynchronous Reliable broadcast protocol from [4] with byzantine failures. In both cases, Theorem 1.7 is used to simplify, clarify and improve the protocols. Similar analyses can be applied to AoN agreement protocols in general. Interesting open problems include using dynamic common knowledge in lower bound proofs, and applying it to the study of non-AoN agreement problems.

Acknowledgments

Yoram Moses was supported by the Israel Pollak academic chair at the Technion. Both his work and that of Or David were supported in part by the Israel Science Foundation under grant 2061/19.

[5] Of course, in binary Consensus protocols we also need $\models \theta_0 \Leftrightarrow \neg\theta_1$.

References

[1] E. A. Akkoyunlu, K. Ekanadham, and R. V. Huber. 1975. Some Constraints and Tradeoffs in the Design of Network Communications. In *Proceedings of the Fifth ACM Symposium on Operating Systems Principles (SOSP '75)*. 67–74.

[2] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. 1995. Sharing memory robustly in message-passing systems. *Journal of the ACM (JACM)* 42, 1 (1995), 124–142.

[3] Michael Ben-Or. 1983. Another advantage of free choice (Extended Abstract): Completely asynchronous agreement protocols. In *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing*.

[4] Gabriel Bracha. 1987. Asynchronous Byzantine agreement protocols. *Information and computation* 75, 2 (1987), 130–143.

[5] Bernadette Charron-Bost and Shlomo Moran. 2021. Stabilizing Consensus in Message-Passing Systems Reachable by Omission Adversaries. *Distributed Computing* 34, 1 (2021), 51–67.

[6] Soma Chaudhuri. 1990. Agreement is harder than consensus: Set consensus problems in totally asynchronous systems. In *Proceedings of the ninth annual ACM symposium on Principles of distributed computing*. 311–324.

[7] Jing Chen and Silvio Micali. 2019. Algorand: A secure and efficient distributed ledger. *Theoretical Computer Science* 777 (2019), 155–183.

[8] Danny Dolev. 1982. The Byzantine generals strike again. *Journal of algorithms* 3, 1 (1982), 14–30.

[9] Danny Dolev, Nancy A Lynch, Shlomit S Pinter, Eugene W Stark, and William E Weihl. 1986. Reaching approximate agreement in the presence of faults. *Journal of the ACM (JACM)* 33, 3 (1986), 499–516.

[10] Danny Dolev and H. Raymond Strong. 1983. Authenticated algorithms for Byzantine agreement. *SIAM J. Comput.* 12, 4 (1983), 656–666.

[11] Cynthia Dwork and Yoram Moses. 1990. Knowledge and common knowledge in a Byzantine environment: crash failures. *Information and Computation* 88, 2 (1990), 156–186.

[12] Cynthia Dwork and Dale Skeen. 1983. The inherent cost of nonblocking commitment. In *Proceedings of the second annual ACM symposium on Principles of distributed computing*. 1–11.

[13] Ronald Fagin, Joseph Y Halpern, Yoram Moses, and Moshe Vardi. 2003. *Reasoning about knowledge*. MIT press.

[14] Pesech Feldman and Silvio Micali. 1997. An optimal probabilistic protocol for synchronous Byzantine agreement. *SIAM J. Comput.* 26, 4 (1997), 873–933.

[15] Michael J Fischer, Nancy A Lynch, and Michael Merritt. 1986. Easy impossibility proofs for distributed consensus problems. *Distributed Computing* 1, 1 (1986), 26–39.

[16] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. 1985. Impossibility of distributed consensus with one faulty process. *Journal of the ACM (JACM)* 32, 2 (1985), 374–382.

[17] Krisztina Fruzsa, Roman Kuznets, and Ulrich Schmid. 2021. Fire! *Electronic Proceedings in Theoretical Computer Science* 335 (2021), 139–153. doi:10.4204/eptcs.335.13

[18] Yannai A. Gonczarowski and Yoram Moses. 2024. Common Knowledge, Regained. arXiv:2311.04374 [econ.TH] https://arxiv.org/abs/2311.04374

[19] Jim Gray. 1978. Notes on Data Base Operating Systems. In *Operating Systems, An Advanced Course*. Lecture Notes in Computer Science, Vol. 60. Springer, 393–481.

[20] Rachid Guerraoui and Luis Rodrigues. 2006. *Introduction to reliable distributed programming*. Springer.

[21] Joseph Y Halpern and Yoram Moses. 1990. Knowledge and common knowledge in a distributed environment. *Journal of the ACM (JACM)* 37, 3 (1990), 549–587.

[22] Maurice Herlihy, Dmitry Kozlov, and Sergio Rajsbaum. 2013. *Distributed computing through combinatorial topology*. Newnes.

[23] Maurice Herlihy and Nir Shavit. 1999. The topological structure of asynchronous computability. *Journal of the ACM (JACM)* 46, 6 (1999), 858–923.

[24] Roman Kuznets, Laurent Prosperi, Ulrich Schmid, and Krisztina Fruzsa. 2019. Epistemic reasoning with byzantine-faulty agents. In *International Symposium on Frontiers of Combining Systems*. Springer, 259–276.

[25] Leslie Lamport. 1983. The weak Byzantine generals problem. *Journal of the ACM (JACM)* 30, 3 (1983), 668–676.

[26] Leslie Lamport. 2001. Paxos made simple. *ACM SIGACT News (Distributed Computing Column) 32, 4 (Whole Number 121, December 2001)* (2001), 51–58.

[27] David K Lewis. 1969. *Convention: A Philosophical Study*. Harvard University Press, Cambridge, Mass.

[28] Nancy A Lynch. 1996. *Distributed algorithms*. Elsevier.

[29] Yoram Moses. 2016. Relating Knowledge and Coordinated Action: The Knowledge of Preconditions Principle. *Electronic Proceedings in Theoretical Computer Science (TARK 2015)* 215 (2016), 231–245. doi:10.4204/eptcs.215.17

[30] Yoram Moses and Mark R Tuttle. 1988. Programming simultaneous actions using common knowledge. *Algorithmica* 3, 1 (1988), 121–169.

[31] Marshall Pease, Robert Shostak, and Leslie Lamport. 1980. Reaching agreement in the presence of faults. *Journal of the ACM (JACM)* 27, 2 (1980), 228–234.